CHROME

WebMCP Early Preview

Authors Alexandra Klepper, Andre Bandarra, Carlos Giraldo, François Beaufort

Contact
- Questions & Feedback
- Bug reports

Updated Apr 24, 2026

Introduction

WebMCP is a proposed web standard that exposes structured tools for AI agents on existing websites. This would replace "screen-scraping" with robust, high-performance page interaction and knowledge retrieval. WebMCP provides JavaScript and annotates HTML form elements so that agentic browsers know exactly how to interact with page features to support a user's experience.

By exposing APIs to the browser agent, WebMCP significantly improves the performance and reliability of AI agent actuation.

The following are some examples of how WebMCP could be applied to your website:

- Example Airline's website exposes a `book_flight` tool. Agents use this tool to submit structured passenger data directly, rather than guessing how data should be input, based on Example Airline's calendar UI which is designed for human users.
- Complex medical or legal portals annotate forms as a `submit_application` tool. The agent can map data to fields correctly and instantly, instead of repeating data in incorrect fields. For example, indicating a field requires a full legal name versus separate first and last name fields.
- A developer settings page exposes a `run_diagnostics` tool, so an agent can trigger fixes that are otherwise hidden behind nested menus.

Purpose

WebMCP bridges the gap between web applications and AI agents by providing a contract for interaction. Rather than the agent guessing the purpose of a button or the structure of a form, the website can explicitly publish its interface. This contract defines:

- **Discovery:** A standard way for agents to query exactly which tools a page supports, such as `checkout` or `filter_results`.
- **JSON Schemas:** Explicit definitions of inputs and expected outputs, to reduce hallucination or misunderstanding.
- **State:** A shared understanding of the current page context, so the agent knows what resources are available to act on in real time.

Early preview goals

Our goal for the WebMCP early preview is to understand the following topics:

- **Use cases**: Identify when you think you'd use WebMCP and how it supports a given task.
- **API shape and usability**: Feedback on the ergonomics of the WebMCP APIs. Are there functions you expect to find in the APIs that don't yet exist?
- **Tooling ecosystem**: Debugging, testing, and verifying tool definitions with evaluations (evals).
- **Implementation issues**: Bugs or friction points in the Chrome implementation. Please note that this is an **early** preview - there will likely be bugs and points of friction, and we're hoping you'll help us find them.

We are seeking your feedback to shape our understanding.

Availability

WebMCP is available behind a flag in Chrome 146. Learn more about Chrome flags.

Requirements

To use WebMCP, you need:

- **Chrome**: Version 146.0.7672.0 or higher.
- **Flags**: The "WebMCP for testing" flag must be enabled.

Setup

1. Open Chrome and navigate to `chrome://flags/#enable-webmcp-testing`
2. Set the flag to **Enabled**.
3. Relaunch Chrome to apply the changes.

Demo

Install the Chrome Extension

Install the Model Context Tool Inspector Extension to see how WebMCP works in the live demo. The extension lets you inspect registered functions, execute them manually or test them with an agent.

List registered tools

The extension acts as a monitoring interface for the `navigator.modelContext` API, with the experimental `navigator.modelContextTesting` API. It provides a list of all registered tools within the active tab.

- It detects tools registered through both imperative methods, such as `registerTool()` and declarative methods using HTML form annotations.
- For each registered tool, the extension displays the name, description, and the input schema (JSON) for that tool.

Call tools manually

One of the goals for this extension is to offer you a way to bypass the non-deterministic nature of an AI agent for initial testing.

- Developers can manually fill in parameters based on the tool's input schema directly within the extension UI.
- Upon execution, the extension triggers the tool's execute function (for imperative tools) or submits the associated form (for declarative tools) and displays the returned content.
- The extension provides detailed error messages and DevTools integration if a tool execution fails, helping to identify schema mismatches or runtime errors.

Talking to the agent

The extension includes Gemini API support to simulate real-world agent interactions.

- Users can enter natural language prompts to see if the agent can correctly identify and invoke the appropriate WebMCP tools.
- It currently supports prompting the `gemini-2.5-flash` model.
- This feature allows developers to optimize their tool descriptions and schemas based on whether the agent triggers the functions with the correct parameters.

Note: This is separate from the Gemini in Chrome features.

Live demo

Explore the WebMCP capabilities with our hosted travel demo:
https://googlechromelabs.github.io/webmcp-tools/demos/react-flightsearch/

See the list of tools registered

1. With the Model Context Tool Inspector Extension installed, navigate to https://googlechromelabs.github.io/webmcp-tools/demos/react-flightsearch/.
2. The extension indicates there's a tool registered on the page:
3. Click on the extension to open it. You will see a list of tools with their names, descriptions and input schema.

What happened: The Model Context Tool Inspector Extension pulled the tools registered with `registerTool` or `providedContext`, with their name, description and arguments. This is useful to debug your implementation and identify if the correct functions are available at a given UI state.

Manually execute the `searchFlights` tool

1. With the Model Context Tool Inspector Extension installed, navigate to https://googlechromelabs.github.io/webmcp-tools/demos/react-flightsearch/ and then open the extension.
2. At the bottom of the UI, the `searchFlights` tool should already be selected. If not already selected, make sure to change the value in the "Tool" combo box to "searchFlights".
3. Add the value below to the "Input Arguments" field.

```JSON
{
   "origin": "LON",
   "destination": "NYC",
   "tripType": "round-trip",
```

```
  "outboundDate": "2026-06-10",
  "inboundDate": "2026-06-17",
  "passengers": 2
}
```

4. Click the “Execute Tool” function.

NOTE: The demo application only supports the LON => NYC flight segment with round-trips. Using different parameters triggers the function, but *won’t show results in the page*.

What happened: The Model Context Tool Inspector Extension executed a tool by calling it directly, using the arguments provided. This is useful to test your function’s implementation, without relying on a large language model to translate natural language input into the function arguments.

Invoke the `searchFlights` tool with natural language

1. With the Model Context Tool Inspector Extension installed, navigate to https://googlechromelabs.github.io/webmcp-tools/demos/react-flightsearch/. Then, open the extension.
2. Use **Set a Gemini API key** to add a Gemini API key and enable the agent. Get your key from AI Studio.
3. Input the text below in the “User Prompt” field and click **Send**.

```
None
Search flights from LON to NYC leaving next Monday and returning after a week for
2 passengers.
```

NOTE: The demo application only supports the LON => NYC flight segment with round-trips. Using different parameters triggers the function, but *won’t show results in the page*.

What happened: The extension prompted Gemini 2.5 Flash with your input, alongside the function definitions using the Function Calling functionality. The model told the extension which function to call and parameters, and the extension called the function.

Demo resources

Examples of demos covering both imperative and declarative implementations are available:

- Travel Demo (React) | WebMCP Imperative Demo
- Le Petit Bistro | WebMCP Declarative Demo
- WebMCP zaMaker! | WebMCP Imperative Demo

You can also review and explore the demo source code on GitHub.

API overview

There are two APIs:

- Imperative API
- Declarative API

Imperative API

Use the Imperative API to define tools, using standard JavaScript.

`registerTool`

Imperative tools can be registered and unregistered dynamically with `registerTool`. Use this method to add a single tool to the existing set without removing others. Unregistering the tool can be done by using the `AbortSignal` passed as an optional parameter.

Set `untrustedContentHint` to `true` if the tool processes data from external or unverified sources to indicate that the tool's output contains data that is untrusted, from the perspective of the author registering the tool.

```javascript
const addTodoTool = {
  name: "addTodo",
  description: "Add a new item to the todo list",
  inputSchema: {
    type: "object",
    properties: { text: { type: "string" } },
  },
  execute: ({ text }) => {
    // You should handle the persistence logic here (omitted for demo)
    return `Added todo: ${text}`;
  },
  annotations: { readOnlyHint: false, untrustedContentHint: true },
};
const controller = new AbortController();
```

```
navigator.modelContext.registerTool(addTodoTool, { signal: controller.signal });

// Unregister the tool later...
controller.abort();
```

Declarative API

Use the Declarative API to automatically transform standard HTML forms into WebMCP tools by adding specific annotations. These annotations define the tool's name and purpose through the `toolname` and `tooldescription` HTML attributes on the `<form>` element, while the form fields act as the parameters for the tool. The browser then translates these elements into a structured representation that an AI agent can interpret and use, similar to how imperative tools are handled.

When an AI agent calls a tool by its registered name through the `toolname` HTML form attribute, the browser automatically brings the associated form into focus and populates its fields. By default, the form remains visible to the user, who must manually click the `<form>`'s **Submit** button to proceed, unless the `toolautosubmit` HTML form attribute is set. The results of the tool invocation are either displayed in the current document, or if the submission triggers a navigation change, in the triggered document.

Form-associated elements support the `toolparamdescription` attribute, which provides an optional way to define a property description within the json-schema. In its absence, the browser uses the `textContent` of the associated `<label>` HTML element but skips descendants that are labelable or, if no label exists, the `aria-description`. However, if you need to attach a `toolparamdescription` to a group of related elements, such as a set of <input type="radio"> buttons, you must place the attribute on the nearest parent <fieldset> element so that it applies to the parameter group as a whole.

```HTML
<form toolname="my_tool" tooldescription="A simple declarative tool"
action="/submit">

  <label for="text">text label</label>
  <input type=text name=text>

  <select name="select" required toolparamdescription="A nice description">
```

```
    <option value="Option 1">This is option 1</option>
    <option value="Option 2">This is option 2</option>
    <option value="Option 3">This is option 3</option>
  </select>

  <button type=submit>Submit</button>
</form>
```

The HTML code above results in the following tool name, description and input schema computed internally by the browser for the declarative tool.

```
JSON
[
  {
    "name": "my_tool",
    "description": "A simple declarative tool",
    "inputSchema": {
      "type": "object",
      "properties": {
        "text": {
          "type": "string",
          "description": "text label"
        },
        "select": {
          "type": "string",
          "anyOf": [
            {
              "const": "Option 1",
              "title": "This is option 1"
            },
            {
              "const": "Option 2",
              "title": "This is option 2"
            },
            {
              "const": "Option 3",
              "title": "This is option 3"
            }
          ],
          "enum": [
            "Option 1",
            "Option 2",
```

```
            "Option 3"
          ],
          "description": "A nice description"
        }
      },
      "required": [
        "select"
      ]
    }
  }
]
```

The `SubmitEvent` interface introduces a new `agentInvoked` boolean attribute. This attribute is set to true whenever a form is triggered by an AI agent, to adapt your web app's behavior specifically for agent-based interactions.

Additionally, the `SubmitEvent` now also includes the `respondWith(Promise<any>)` method which lets you pass a promise to the browser that you resolve with the form's results data. The resulting value is then serialized and returned to the model as the tool's output. To use it, you must first call `preventDefault()` to stop the browser's standard form submission.

```HTML
<form toolautosubmit
      toolname="search_tool"
      tooldescription="Search the web" action="/search">
  <input type=text name=query>
</form>
<script>
  document.querySelector("form").addEventListener("submit", (e) => {
    e.preventDefault();

    if (!myFormIsValid()) {
      if (e.agentInvoked) { e.respondWith(myFormValidationErrors) };
      return;
    }

    if (e.agentInvoked) { e.respondWith("Search is done!"); }
  });
</script>
```

The browser signals that an AI agent executed a tool via the `"toolactivated"` event, which fires on the window once form fields are pre-filled. Conversely, if a user cancels the agentic operation or the `reset()` method is invoked on the HTML form, a `"toolcancel"` event is triggered. Both of these events are non-cancelable and provide a `toolName` attribute for identification.

```javascript
window.addEventListener('toolactivated', ({ toolName }) => {
  console.log(`the tool "${toolName}" execution was activated.`);
  // TODO: Update UI or validate form if needed.
});

window.addEventListener('toolcancel', ({ toolName }) => {
  console.log(`the tool "${toolName}" execution was cancelled.`);
  // TODO: Let the user know. Update UI.
});
```

When an AI agent successfully invokes a tool, focuses the associated form, and auto-populates its fields, the browser triggers specific CSS pseudo-classes for visual feedback:

- `:tool-form-active` is applied to the tool's HTML `form` element.
- `:tool-submit-active` is applied to the form's submit button, if one is present.

Both are deactivated when the form submits, the AI agent cancels the action, or the user resets the form.

The following placeholder styles are applied by default in Chrome.

```css
form:tool-form-active {
  outline: light-dark(blue, cyan) dashed 1px;
  outline-offset: -1px;
}

input:tool-submit-active {
  outline: light-dark(red, pink) dashed 1px;
  outline-offset: -1px;
}
```

Tool declaration best practices

A well-structured tool declaration should be self-explanatory, so developers understand how it works and can use it immediately, without having to look at the outputs and retry. Here are some best practices that make this possible:

1. Naming and semantics

Goal: Ensure clarity of scope and intended use.

- **Principle: Distinguish execution from initiation**
  - Use specific verbs that describe exactly what happens.
  - *Example:* Use `create-event` for immediate creation, but `start-event-creation-process` if the tool redirects the user to a UI form.
- **Principle: Positive and clear descriptions.**
  - Describe *what* the tool does and *when* to use it.
  - Strongly prefer positive instructions over negative limitations.
  - *Good Example:* "This tool can create a calendar event, scheduled for a specific date and time..."
  - *Bad Example:* "Do not use for weather..." (Limitations should be implicit in a good description).

2. Schema design and handling input

Goal: Minimize cognitive computing for the model.

- **Principle: Accept raw user input.**
  - Avoid asking the agent to perform math or transformations.
  - *Example:* If a user says "11:00 to 15:00", the tool should accept strings `11:00` and `15:00`, not require the model to calculate minutes-from-midnight (e.g., `660`).
- **Principle: Explicit types and business logic.**
  - All parameters must have specific types (`string`, `number`, `enum`).
  - Explain the *why* behind options, not just the *what*.
  - *Example:* "Use `shipping='EXPRESS'` when the user requests next-day delivery, rather than an ambiguous `shipping_id=1`.

3. Reliability and error recovery

Goal: Turn failures into corrections.

- **Principle: validate strictly in code, loosely in schema.**
  - While schema constraints are helpful, they are not guaranteed.
  - Validate constraints within your function code and return **descriptive errors**. This allows the model to self-correct and retry with valid parameters.

- **Principle: Graceful failure for rate limits.**
    - Tools should allow for reasonable repeated use (e.g., comparing prices).
    - If rate limited, return a meaningful error or advise the user to take over manually.
- **Principle: Return after UI has been updated**
    - Agents might use UI state to verify function execution and plan next steps.
    - Ensure the function returns *after* UI updates for consistency.

4. Tool strategy and philosophy

Goal: Create non-overlapping, composable tools.

- **Principle: Atomic and composable.**
    - Avoid similar tools with nuanced differences. Combine them into a single tool with input parameters.
    - Each tool should be a single function.
- **Principle: Trust the agent's flow control.**
    - Avoid rigid instructions like "Don't call B after A".
    - It is not the tool's responsibility to manage the global task flow.
- **Principle: Tool registration management**
    - Imperative tools can be registered and unregistered dynamically with `registerTool`.
    - With the declarative API, a tool can be dynamically registered or unregistered by adding or removing the `toolName` and `toolDescription` attributes on a form.
    - Register tools when they make sense for a page state, and unregister when they don't make sense anymore.

Limitations

- **Browsing context required**: Since tool calls are handled in JavaScript, a browsing context (i.e. a browser tab or a webview) must be opened. There is no support for agents or assistive tools to call tools "headlessly," meaning without visible browser UI.
- **UI synchronization**: For a satisfactory end user experience, web developers must ensure their UI is updated to reflect the current app state, regardless of whether the state updates came from human interaction or from a tool call.
- **Complexity overhead**: In cases where the site UI is very complex, developers will likely need to refactor or add JavaScript that handles app and UI state with appropriate outputs.
- **Tool discoverability**: There is no built-in mechanism for client applications to discover which sites provide callable tools without visiting or querying them directly. Search engines, or directories of some kind, may play a role in helping client applications

determine whether a site has relevant tools for the task it is trying to perform. We are actively exploring this space and cannot make any commitments at this time.

Provide feedback

- **Share your questions and comments** in the Dev Preview Group.
- **File bugs**: https://crbug.com/new?component=2021259

Frequently asked questions

What is the difference between MCP and WebMCP?

The Model Context Protocol (MCP) enables AI Agents to connect to applications with a server-side protocol. Developers who want to give agents access to their tools need to deploy them on their own server.

WebMCP is a protocol inspired by MCP that allows developers to provide tools to in-browser AI agents. WebMCP helps developers implement client-side functions or use a declarative API in their existing web application, so agents are better equipped to interact with the application.

Can I use WebMCP in other browsers?

WebMCP is a proposed web standard. Our goal is to build APIs that any browser with agentic capabilities can implement and benefit from. The early preview is only in Chrome.

You can follow along this process on GitHub.

Changelog

DATE	CHANGES
Apr 24, 20...	- Added `readOnlyHint` and `untrustedContentHint` annotations (source)
Mar 27, 20...	- Added `registerTool()` method `signal` option and removed `unregisterTool()` (source) - Updated input groups (radio, etc) `toolparamdescription` using nearest <fieldset> (source) - Switched from `oneOf` to `anyOf` in the computed input schema example. (source)
Mar 12, 20...	- Removed `toolparamtitle` attribute

	- Added demo
Mar 6, 2026	- Removed `provideContext` and `clearContext` methods
Feb 10, 20...	- Declarative API tweaks - Added Demo resources section
Feb 9, 2026	- Initial version